

ALL-IN-RAG / CODE C8

食谱 RAG 系统 面试准备文档

基于 `all-in-rag/code/C8` 目录整理，覆盖项目介绍、架构链路、模块设计、检索优化、生成策略、工程化亮点、面试高频问答与可扩展方向。目标是帮助你在面试中既能讲清楚项目，又能接住 RAG 技术追问。

项目主题	本地食谱知识库问答与菜品推荐 RAG 系统
核心技术	LangChain、FAISS、BGE Embedding、BM25、RRF、Qwen/通义千问、Chainlit
重点能力	结构化分块、混合检索、元数据过滤、父子文档映射、多轮上下文改写、查询路由、流式输出
适用场景	算法工程、LLM 应用工程、RAG 项目经验、AI Agent/对话系统相关面试

目录

1. 项目一句话介绍
2. 项目目录与模块职责
3. 整体架构与请求链路
4. 数据处理设计
5. 索引构建与向量化
6. 检索优化设计
7. 生成模块与 Prompt 设计
8. 多轮对话与查询改写
9. 查询路由设计
10. 工程化能力
11. 面试高频问题与参考回答
12. 项目亮点总结
13. 不足与优化方向
14. 面试讲述稿

本文档按“能讲项目”和“能答追问”两条线组织。前半部分用于建立完整技术叙事，后半部分用于面试即时回答。

1. 项目一句话介绍

30 秒版本

这个项目是一个面向本地食谱知识库的 RAG 问答系统。系统把 Markdown 格式的菜谱文档解析成结构化 chunk，使用 BGE 模型做向量化并存入 FAISS，同时结合 BM25 关键词召回和 RRF 重排提升检索质量。用户可以通过 命令行或 Chainlit Web UI 提问，比如“宫保鸡丁怎么做”“推荐几个简单素菜”，系统会根据问题类型进行 查询路由、上下文改写、混合检索，并调用大模型生成可操作的回答。

2 分钟版本

这个项目主要解决的是“如何基于本地私有知识库做准确、可解释、可交互的问答”。我选择食谱作为垂直领域数据，因为菜谱天然包含菜名、分类、难度、食材、步骤、技巧等结构化信息，非常适合验证 RAG 的完整链路。

在数据侧，系统读取本地 Markdown 文档，按标题层级切分，并为每个文档和 chunk 增强菜品分类、菜名、难度等元数据。在索引侧，系统使用 BAAI/bge-small-zh-v1.5 生成文本向量，存入本地 FAISS 索引，并支持本地持久化加载。在检索侧，系统同时使用向量检索和 BM25 检索，然后通过 RRF 进行融合重排，兼顾语义匹配和关键词匹配。在生成侧，系统先用 query router 判断问题是列表推荐、详细步骤还是通用问题，再选择不同的生成策略。

另外，项目还做了多轮对话记忆和 query rewrite，可以处理“这个怎么做”“它需要什么食材”这类省略主语的问题。Chainlit 前端使用全局单例复用知识库，支持流式输出，提升交互体验。

5 分钟版本

面试中如果需要展开，可以按“背景、方案、难点、效果、改进”五段讲。

1. **背景**：传统大模型缺少本地食谱数据，容易出现编造；而直接全文搜索又无法理解语义，所以引入 RAG。
2. **方案**：构建“数据准备 - 索引构建 - 检索优化 - 生成集成 - Web UI”的完整链路。
3. **难点**：分块如何保留菜谱结构、检索如何兼顾菜名关键词和语义表达、多轮对话如何补全上下文。
4. **效果**：系统可以回答具体做法、食材问题、推荐列表，并支持本地缓存和索引持久化，启动速度更快。
5. **改进**：后续可加入 reranker、评测集、Milvus、GraphRAG、多模态食材识别和生产级监控。

2. 项目目录与模块职责

文件/目录	职责	面试可讲点
main.py	系统主流程编排，负责初始化模块、构建知识库、执行问答流程。	体现模块化设计和端到端 RAG pipeline。
config.py	集中管理数据路径、索引路径、embedding 模型、LLM 参数、top_k 等配置。	体现工程化配置管理。
rag_modules/data_preparation.py	加载 Markdown、提取元数据、结构化分块、父子映射、缓存与增量检测。	这是 RAG 效果的基础，分块和元数据质量直接影响检索质量。
rag_modules/index_construction.py	初始化 embedding 模型，构建、保存、加载 FAISS 向量索引。	可讲向量化、归一化、索引持久化和本地部署。
rag_modules/retrieval_optimization.py	实现向量检索、BM25 检索、RRF 融合重排、元数据过滤。	这是项目的核心亮点之一，可以重点讲混合检索的必要性。
rag_modules/generation_integration.py	负责 LLM 调用、Prompt 设计、query rewrite、query router、流式生成。	体现 RAG 与 LLM 生成策略的结合。
web_ui_recipe.py	Chainlit Web UI，处理会话开始、用户消息、流式输出和多轮历史。	体现产品交互、会话状态管理和应用落地能力。
vector_index/	保存本地 FAISS 索引文件，如 index.faiss 和 index.pkl。	说明系统支持本地持久化，避免每次启动全量重建。

3. 整体架构与请求链路

用户提问

-> Chainlit / CLI 接收问题

-> Query Router 判断问题类型: list / detail / general

-> Query Rewrite 结合历史对话补全省略信息

-> 提取元数据过滤条件: 分类、难度等

-> 混合检索:

1. FAISS 向量语义召回

2. BM25 关键词召回

-> RRF 融合重排

-> 从子 chunk 找回父文档

-> 构造上下文 context

-> 根据问题类型选择 Prompt

-> LLM 生成回答

-> 流式返回给用户

为什么这个链路比较完整

很多 RAG demo 只做到“切分文档、向量检索、拼 Prompt”，但实际面试中容易被追问检索质量、多轮上下文、缓存、工程化和用户体验。这个项目覆盖了这些常见追问点：

- 检索质量：不只依赖向量检索，而是加入 BM25 和 RRF。
- 上下文质量：使用父子文档设计，检索小 chunk，生成时回到完整菜谱。
- 用户体验：支持 Chainlit UI、流式输出和多轮记忆。
- 工程效率：索引和数据解析状态都支持本地持久化。

核心对象关系

对象	作用
父文档 parent document	完整菜谱文档，适合用于最终生成回答，保证上下文完整。
子文档 chunk	按 Markdown 标题切出来的小片段，适合用于检索，提高匹配精度。
元数据 metadata	包含菜名、分类、难度、source、parent_id、chunk_id 等字段，用于过滤和回溯。
向量索引 vectorstore	保存 chunk 的语义向量，支持相似度检索。

4. 数据处理设计

4.1 为什么直接读取 Markdown

食谱数据本身是 Markdown 文件，包含标题、二级标题、步骤、食材等结构。直接读取 Markdown 可以最大程度保留原始格式，避免通用文本抽取工具破坏层级结构。对于 RAG 来说，文档结构本身就是重要语义。

4.2 Markdown 结构化分块

系统使用 `MarkdownHeaderTextSplitter` 按 `#`、`##`、`###` 等标题层级切分文档。这样做比固定长度切分更适合菜谱，因为“食材”“步骤”“技巧”等内容通常天然分布在不同标题下。

面试表达：固定长度切分虽然简单，但容易把一个步骤切断，或者把多个无关段落混在一起。按 Markdown 标题切分能保留业务语义边界。

4.3 元数据增强

系统会在加载文档时增强元数据：

- `dish_name`：从文件名提取菜品名称。
- `category`：从路径中的目录名映射出荤菜、素菜、汤品、甜品、早餐、主食、水产、调料、饮品等分类。
- `difficulty`：从文档内容中的难度标记推断难度等级。
- `parent_id`：基于相对路径生成稳定 ID，用于父子文档映射。
- `chunk_id`：为每个子 chunk 生成唯一 ID。

4.4 父子文档映射

项目中检索的是 chunk，但最终生成时会通过 `parent_id` 找回完整父文档。这是一个很有价值的设计：

- 检索阶段使用小 chunk，提高匹配精度。
- 生成阶段使用完整文档，避免上下文缺失。
- 多个 chunk 命中同一个菜谱时，可以按命中次数估计父文档相关性。

4.5 缓存与增量检测

数据准备模块会把解析后的 documents、chunks、parent_child_map 和 file_hashes 保存到本地缓存。启动时优先加载缓存，并通过文件修改时间检测新增或修改的 Markdown 文件，从而避免每次启动都重新解析所有文档。

面试时可以补充：当前增量更新更偏“工程优化雏形”，如果要生产级，需要处理删除文件、精确更新旧 chunk、索引删除和版本一致性。

5. 索引构建与向量化

5.1 Embedding 模型选择

项目默认使用 `BAAI/bge-small-zh-v1.5`。这个模型对中文语义检索比较友好，模型体积也相对适合本地运行。对食谱这种中文领域知识库来说，中文 embedding 模型比通用英文 embedding 更合适。

5.2 向量归一化

索引构建时设置了 `normalize_embeddings=True`。向量归一化可以让相似度计算更稳定，尤其是在使用余弦相似度思想时，归一化后的向量长度一致，模型更关注方向上的语义相似。

5.3 FAISS 本地索引

系统使用 FAISS 存储 chunk 向量。FAISS 的优势是本地部署简单、查询速度快、适合中小规模知识库实验和单机应用。在 C8 项目中，索引会保存到 `vector_index/` 目录，后续启动可以直接加载。

5.4 面试追问：为什么不用 Milvus

可以这样回答：C8 这个项目是本地知识库和教学实验场景，数据规模不大，用 FAISS 可以减少部署复杂度，快速验证 RAG 链路。如果进入生产或多用户场景，数据量变大、需要分布式扩展、权限隔离、在线更新和监控，就可以升级到 Milvus、Qdrant 或 Elasticsearch 向量检索方案。

6. 检索优化设计

6.1 为什么不用纯向量检索

纯向量检索擅长语义相似，但在菜名、食材名、专有名词、短关键词上可能不如关键词检索稳定。比如“鱼香肉丝”“宫保鸡丁”这类菜名，BM25 的精确词项匹配很有价值；而“想吃清淡一点的菜”这类表达，则向量检索更有优势。

6.2 混合检索方案

项目的混合检索包含两路召回：

1. **向量检索**：通过 FAISS 找语义相似 chunk。
2. **BM25 检索**：通过词频和逆文档频率找关键词相关 chunk。

两路召回后，系统用 RRF 进行融合重排。

6.3 RRF 融合重排

RRF，全称 Reciprocal Rank Fusion，是一种基于排名的融合方法。它不直接依赖不同检索器的原始分数，因为向量相似度和 BM25 分数通常不在同一个尺度上。RRF 更关注文档在各个检索结果中的排名。

$$\text{RRF score} = \sum \frac{1}{k + \text{rank}}$$

如果一个 chunk 在向量检索和 BM25 检索中排名都靠前，它的最终得分就更高。这样可以提升结果稳定性。

6.4 元数据过滤

系统会从用户问题中提取分类和难度关键词。例如用户问“推荐几个简单素菜”，系统可以识别出：

```
filters = {  
  "category": "素菜",  
  "difficulty": "简单"  
}
```

然后先扩大召回候选集，再按元数据过滤。这样比单纯依赖 prompt 让模型自己筛选更稳定，也更可解释。

6.5 面试表达

这个项目中的检索优化可以总结为一句话：用向量检索覆盖语义泛化，用 BM25 保证关键词精确命中，用 RRF 解决不同检索器分数不可比问题，再用元数据过滤处理明确约束条件。

7. 生成模块与 Prompt 设计

7.1 生成模块职责

`GenerationIntegrationModule` 负责调用大模型，并根据问题类型生成不同风格的回答。项目中使用 `ChatTongyi` 接入通义千问模型，API Key 从 `DASHSCOPE_API_KEY` 环境变量读取。

7.2 三类回答策略

问题类型	处理方式	示例
<code>list</code>	直接基于父文档提取菜品名称，返回推荐列表。	“推荐几个简单素菜”
<code>detail</code>	使用步骤型 Prompt，生成菜品介绍、食材、制作步骤和技巧。	“宫保鸡丁怎么做”
<code>general</code>	使用基础问答 Prompt，围绕检索上下文回答。	“什么是川菜”

7.3 Prompt 设计重点

- 明确角色：专业烹饪助手或烹饪导师。
- 明确输入：用户问题和检索到的食谱上下文。
- 明确约束：根据资料回答，信息不足时要说明。
- 明确输出结构：详细步骤类问题建议输出菜品介绍、食材、步骤、技巧。

7.4 上下文长度控制

`_build_context` 会把检索到的文档拼成上下文，并限制最大长度。这个设计可以防止一次塞入过多文档导致 prompt 太长，同时减少无关上下文对模型回答的干扰。

7.5 如何降低幻觉

本项目主要通过三种方式降低幻觉：第一，先检索再生成，让回答基于本地食谱；第二，在 Prompt 中要求信息不足时诚实说明；第三，列表类问题尽量不让模型自由发挥，而是从检索到的文档中提取菜名返回。

8. 多轮对话与查询改写

8.1 为什么需要 query rewrite

多轮对话中，用户经常省略主语。例如：

用户：红烧肉怎么做？
助手：.....
用户：它需要哪些食材？

如果直接把“它需要哪些食材？”拿去检索，系统不知道“它”指什么。query rewrite 的作用就是结合历史对话，把问题改写成“红烧肉需要哪些食材？”。

8.2 历史对话管理

Chainlit 前端使用 `cl.user_session` 保存每个用户的 `chat_history`。每轮对话结束后，把用户问题和助手回答追加进去，并限制最多保留最近若干条消息，避免 token 过长。

8.3 为什么不是所有问题都改写

项目中对 `list` 类型问题会跳过改写，避免推荐类问题被历史上下文错误绑定。对于 `detail` 和 `general` 类型问题，则会结合历史对话做改写。

8.4 面试表达

多轮对话不是简单把历史消息拼进 Prompt，而是需要判断当前问题是否依赖上下文，并把省略或指代的信息转成可检索的明确查询。否则检索阶段就会失败，后续生成再强也无法弥补。

9. 查询路由设计

9.1 路由类型

系统通过 LLM 判断用户问题属于 `list`、`detail` 或 `general`。

路由	含义	后续策略
<code>list</code>	用户想要菜品列表或推荐。	返回菜名列表，减少长篇生成。
<code>detail</code>	用户想知道具体做法、步骤、食材。	找回父文档，生成结构化详细回答。
<code>general</code>	普通解释或泛化问题。	使用基础问答 Prompt。

9.2 为什么需要路由

不同问题需要不同输出。如果用户只是要“推荐几个素菜”，系统没必要生成每道菜的完整做法；如果用户问“鱼香肉丝怎么做”，就应该返回详细步骤。路由可以让系统更符合用户意图，也能减少无效 token 消耗。

9.3 路由失败怎么办

代码中如果路由结果不属于预设类型，会默认回退为 `general`。这是一个简单但实用的兜底策略。生产环境可以进一步加入规则校验、置信度判断或人工可观测日志。

10. 工程化能力

10.1 配置管理

`RAGConfig` 使用 `dataclass` 管理数据路径、索引路径、模型名、`top_k`、`temperature`、`max_tokens` 等参数。这种设计方便后续切换模型、调整召回数量或迁移数据目录。

10.2 环境变量

系统通过 `DASHSCOPE_API_KEY` 读取通义千问 API Key。这样不会把密钥写死在代码里，符合基本安全实践。

10.3 索引持久化

FAISS 索引保存到本地 `vector_index`，启动时优先加载。这样可以避免每次启动都重新 `embedding` 全部文档。

10.4 数据缓存

数据准备模块会把解析状态保存为 `pickle` 缓存，包括父文档、子 `chunk`、父子映射和文件修改记录。当本地文档没有变化时，可以快速恢复知识库状态。

10.5 Web UI 全局单例

`web_ui_recipe.py` 中使用 `global_rag_system`，首次打开时初始化知识库，后续用户复用内存中的系统实例。这能避免每个用户会话都重复加载模型和索引。

10.6 流式输出

生成模块提供 `stream` 版本方法，Chainlit 前端通过 `stream_token` 持续输出片段。用户不需要等待完整回答生成完才看到结果，交互体验更接近真实产品。

11. 面试高频问题与参考回答

Q1: 为什么这个项目适合用 RAG, 而不是直接问大模型?

因为问题依赖本地食谱知识库。直接问大模型可能不知道具体数据, 也容易编造。RAG 可以先从本地菜谱中检索相关内容, 再让大模型基于检索结果回答, 既能利用大模型的表达能力, 又能让答案受控于本地知识。

Q2: 你的分块策略是什么? 为什么这么做?

我使用 Markdown 标题结构分块, 而不是纯固定长度分块。因为菜谱文档天然有“食材”“步骤”“技巧”等标题结构。按标题切分可以保留语义边界, 检索时更容易命中具体片段, 生成时再通过 parent_id 回到完整菜谱, 兼顾精度和完整性。

Q3: chunk 太大或太小会有什么问题?

chunk 太大会混入很多无关内容, 检索精度下降, 也会占用更多上下文窗口。chunk 太小则可能丢失必要上下文, 比如只有一个步骤但没有菜名和食材。所以这个项目按 Markdown 结构切分, 并保留标题, 同时用父子文档映射补回完整上下文。

Q4: 为什么要做混合检索?

向量检索擅长语义匹配, 比如“清淡一点的菜”; BM25 擅长关键词和专有名词匹配, 比如具体菜名或食材名。食谱场景同时存在这两类问题, 所以混合检索比单一路径更稳。

Q5: RRF 是什么? 为什么不用直接加权分数?

RRF 是 Reciprocal Rank Fusion, 它根据文档在不同召回列表中的排名计算融合分数。不同检索器的原始分数尺度不同, 向量相似度和 BM25 分数不能直接相加。RRF 基于排名融合, 可以避免分数归一化困难, 工程上简单稳定。

Q6: 为什么要做父子文档映射?

检索阶段需要小粒度 chunk 提高命中精度, 但生成阶段需要完整菜谱保证上下文完整。父子文档映射让系统可以“用小块检索, 用全文生成”, 避免模型只看到局部片段导致回答不完整。

Q7: 如何处理“这个怎么做”这类多轮问题?

Chainlit 会保存最近几轮对话历史。后端在检索前调用 query rewrite, 把省略主语或代词指代补全成明确查询。比如用户先问“红烧肉是什么”, 再问“它怎么做”, 系统会改写成“红烧肉怎么做”再检索。

Q8: 如何降低大模型幻觉?

首先通过检索把上下文限定在本地食谱文档中; 其次 Prompt 明确要求基于资料回答, 信息不足时说明; 第三, 对于推荐列表类问题, 尽量从检索到的文档元数据中提取菜名, 而不是让模型自由生成。

Q9: 如果数据规模扩大, 该怎么优化?

可以从几个方向优化: 用 Milvus 等向量数据库替代本地 FAISS; 增加离线构建和在线增量更新流程; 加入 reranker 做二阶段排序; 建立检索评测集观察 recall、MRR、nDCG; 对元数据建立更严格的结构化索引。

Q10: 怎么评价这个 RAG 系统效果?

可以分检索和生成两层评估。检索层看 recall@k、MRR、命中文档是否正确; 生成层看答案是否忠实于文档、是否完整、是否可操作。食谱场景还可以人工构造问题集, 比如菜名查询、食材查询、推荐查询和多轮追问。

12. 项目亮点总结

1. **结构化分块**: 利用 Markdown 标题切分, 符合菜谱文档天然结构, 比固定长度切分更贴合业务。
2. **父子文档设计**: 小 chunk 负责精确检索, 父文档负责完整生成, 解决检索精度和上下文完整性的矛盾。
3. **混合检索**: 向量检索加 BM25, 兼顾语义泛化和关键词精确匹配。
4. **RRF 重排**: 基于排名融合不同检索器结果, 避免分数尺度不可比问题。
5. **元数据过滤**: 支持按分类、难度等业务属性约束检索结果。
6. **多轮上下文改写**: 解决省略主语和代词指代问题, 提升连续对话能力。
7. **查询路由**: 根据问题类型选择列表、详细步骤或通用回答策略。
8. **工程化落地**: 支持本地索引持久化、数据缓存、Chainlit UI、流式输出和全局实例复用。

13. 不足与优化方向

13.1 当前不足

- **缺少系统评测**：目前没有固定测试集，难以量化检索和生成效果。
- **没有 reranker**：RRF 之后还可以加入 cross-encoder reranker 提升精排质量。
- **增量更新还不够完整**：当前主要检测新增和修改，生产级还要处理删除、更新旧向量和索引一致性。
- **元数据抽取偏规则**：复杂食材、口味、烹饪方式还没有结构化抽取。
- **缺少异常观测**：线上系统还需要日志、监控、用户反馈和 trace。

13.2 可扩展方向

方向	做法	价值
Reranker	召回 top 20/50 后用 cross-encoder 精排。	提升最终上下文相关性。
Milvus	替换本地 FAISS，支持更大规模和服务化部署。	提升生产可扩展性。
GraphRAG	抽取食材、菜品、做法、口味、菜系等实体关系。	支持复杂条件推荐和多跳推理。
多模态	支持用户上传食材图片或菜品图片。	从文本问答扩展到视觉理解。
评测体系	构造标准问题集，统计 recall@k、MRR、忠实性和完整性。	让优化有量化依据。
用户反馈闭环	收集点赞、点踩、失败查询和低质量回答。	持续改进检索和 Prompt。

14. 面试讲述稿

14.1 开场讲法

我做过一个本地食谱知识库 RAG 问答系统，目标是让用户可以基于本地 Markdown 菜谱进行问答和推荐。这个项目不是简单的向量检索 demo，而是做了比较完整的 RAG 链路，包括数据解析、结构化分块、FAISS 索引、BM25 和向量混合检索、RRF 重排、元数据过滤、父子文档映射、多轮 query rewrite、查询路由以及 Chainlit 流式 Web UI。

14.2 技术难点讲法

我觉得比较关键的难点有三个。第一是分块策略，食谱文档有明确的 Markdown 结构，所以我没有采用简单固定长度切分，而是按标题切分，并通过父子文档映射解决小块检索和完整生成之间的矛盾。第二是检索质量，单纯向量检索对菜名、食材名这类关键词不一定稳定，所以我加入了 BM25，并用 RRF 融合两路召回。第三是多轮对话，用户常常会问“这个怎么做”，所以我加入 query rewrite，在检索前把省略信息补全。

14.3 项目价值讲法

这个项目体现了我对 RAG 的理解：RAG 的核心不只是把文档塞进向量库，而是围绕数据质量、检索质量、上下文构造和生成控制做系统设计。同时项目也考虑了工程化，比如索引持久化、数据缓存、全局实例复用和流式输出。

14.4 被追问时的展开方向

- 如果面试官问数据处理，就重点讲 Markdown 分块、元数据增强和父子文档映射。
- 如果问检索，就重点讲向量检索、BM25、RRF 和元数据过滤。
- 如果问生成，就重点讲 query router、不同 Prompt、上下文长度控制和降低幻觉。
- 如果问工程化，就重点讲缓存、索引持久化、环境变量、Chainlit 会话管理和流式输出。
- 如果问优化，就重点讲 reranker、评测集、Milvus、GraphRAG 和多模态。

14.5 最值得强调的三点

1. 我没有只做单一路径向量检索，而是做了混合检索和 RRF 融合。
2. 我没有只关注模型生成，而是从数据分块、检索、上下文和生成策略全链路优化。
3. 我考虑了真实交互体验，包括多轮记忆、query rewrite、流式输出和 Web UI 复用知识库。

最后收束可以说：如果把这个项目继续推进到生产环境，我会优先补评测集和 reranker，因为没有评测就很难判断优化是否真的有效；然后把 FAISS 升级为服务化向量数据库，并完善在线增量更新和日志观测。